

Agent Skills in der professionellen Softwareentwicklung: Entwurf, Einsatz und Absicherung von KI-Skills in großen Projekten

Lucas Tuscher
Student

Abstract—KI-Coding-Agenten scheitern in großen Codebasen selten an fehlender Sprachkompetenz, sondern an fehlendem Projektwissen: Architekturregeln, Konventionen, Qualitätsmaßstäbe und Prüfwerkzeuge. *Agent Skills* – Ordner mit einer `SKILL.md`, optionalen Referenzen und Skripten, die ein Agent bei Bedarf lädt – sind seit Ende 2025 ein offener, werkzeübergreifender Standard für genau dieses Wissen. Diese Arbeit fasst die aktuelle empirische Evidenz zu Wirksamkeit, Aktivierung, Kontextkosten, Rückkopplungsschleifen und Sicherheit von Skills zusammen und leitet daraus einen praxisnahen Leitfaden ab. Kuratierte Skills erhöhen die Erfolgsrate von Agenten im Mittel um 16,2 Prozentpunkte, selbstgenerierte Skills dagegen nicht; ein Skill wird jedoch in bis zu 56 % der passenden Fälle gar nicht aufgerufen, lange Regeldateien senken die Erfolgsrate und erhöhen die Kosten um über 20 %, und 26,1 % öffentlich verfügbarer Skills enthalten Sicherheitsschwachstellen. Eine Fallstudie an einer C++-Spiel-Engine mit rund 676.000 Codezeilen zeigt, wie sich diese Befunde in eine Architektur aus kurzem Immer-Kontext, fokussiertem Skill, deterministischen Prüfskripten und einem automatischen Hook übersetzen lassen; eine Vergleichsmessung ergab 20 von 20 erfüllten Prüfkriterien mit Skill gegenüber 17 von 20 ohne, bei 23 % höherem Tokenverbrauch. Abschließend bietet die Arbeit einen direkt übernehmbaren Leitfaden mit Entscheidungshilfe, Verzeichnisstruktur, Vorlage, Regeln für Skripte, Anti-Mustern und Freigabe-Checkliste.

Index Terms—Agent Skills, KI-Coding-Agenten, Context Engineering, Software-Qualität, Large Language Models

I. EINLEITUNG

Der Nutzen KI-gestützter Softwareentwicklung ist empirisch widersprüchlich. Kontrollierte Experimente berichten von 55,8 % schnellerer Bearbeitung einer abgegrenzten Aufgabe [1], rund 21 % Zeitersparnis in einem unternehmensinternen Experiment bei Google [2] und 26 % mehr abgeschlossenen Aufgaben in drei Feldexperimenten mit fast 5.000 Entwickler:innen [3]. Demgegenüber benötigten erfahrene Open-Source-Entwickler:innen in einer randomisierten Studie mit KI-Werkzeugen 19 % *mehr* Zeit für Aufgaben in ihren eigenen, ausgereiften Projekten – obwohl sie selbst eine Beschleunigung von 20 % wahrnahmen [4]. Der DORA-Bericht 2025 beschreibt KI daher als *Verstärker*: Sie vergrößert die Stärken guter Organisationen ebenso wie die Schwächen schlechter und erhöht weiterhin die Instabilität von Auslieferungen [5]. Auf Code-Ebene beobachtet GitClear seit der Verbreitung von KI-Assistenten eine stark wachsende Menge duplizierten Codes bei gleichzeitig sinkendem Anteil an Refactoring [6].

Ein gemeinsamer Nenner dieser Befunde ist der Kontext: Generische Modelle kennen die Regeln, Abstraktionen und Fallstricke eines großen Projekts nicht. Genau hier setzen *Agent Skills* an. Sie verpacken prozedurales Projektwissen so, dass ein Agent es erst dann lädt, wenn eine Aufgabe es erfordert [7]. Diese Arbeit beantwortet vier Fragen:

- **RQ1:** Wie funktionieren Agent Skills technisch, und wie grenzen sie sich von Regeldateien, Hooks und Werkzeugen ab?
- **RQ2:** Was sagt die empirische Evidenz über ihre Wirksamkeit und ihre Grenzen?
- **RQ3:** Welche Entwurfsprinzipien folgen daraus für Skills in großen Projekten?
- **RQ4:** Wie setzt man Skills in Unternehmen verlässlich, kosteneffizient und sicher ein?

Der Beitrag besteht aus drei Teilen: einer Synthese aktueller Studien (2023–2026), einer Fallstudie an einer C++-Spiel-Engine einschließlich einer kleinen Vergleichsmessung mit und ohne Skill und einem praxisnahen Leitfaden (Section VI), der als interne Richtlinie für Teams dienen kann. Wer Skills nur bauen will, kann direkt mit dem Leitfaden beginnen; die vorangehenden Abschnitte begründen seine Regeln.

II. GRUNDLAGEN

A. Aufbau eines Skills

Ein Skill ist ein Verzeichnis mit einer Datei `SKILL.md`. Sie beginnt mit einem YAML-Kopf, der mindestens `name` (höchstens 64 Zeichen, Kleinbuchstaben, Ziffern und Bindestriche, identisch mit dem Verzeichnisnamen) und `description` (höchstens 1.024 Zeichen) enthält; darunter folgen die Anweisungen in Markdown [8]. Optional kommen `scripts/` (ausführbarer Code), `references/` (Nachschlagewissen) und `assets/` (Vorlagen, Daten) hinzu. Das Format wurde von Anthropic entwickelt, als offener Standard veröffentlicht und bis Anfang 2026 von über 25 Produkten übernommen, darunter Claude Code, OpenAI Codex, Cursor, GitHub Copilot und Gemini CLI. Werkzeugübergreifend hat sich das Verzeichnis `.agents/skills/` etabliert, Claude Code liest `.claude/skills/`.

B. Progressive Offenlegung

Skills werden in drei Stufen geladen [7], [8]:

- 1) **Metadaten** (ca. 100 Token je Skill): Name und Beschreibung aller Skills stehen dauerhaft im Kontext.

Sie sind die einzige Grundlage, auf der der Agent entscheidet, ob er einen Skill öffnet.

- 2) **Anweisungen** (empfohlen unter 5.000 Token bzw. 500 Zeilen): Der Rumpf der `SKILL.md` wird geladen, wenn die Aufgabe passt.
- 3) **Ressourcen** (unbegrenzt): Referenzdateien werden nur bei Bedarf gelesen; Skripte werden ausgeführt, ohne dass ihr Quelltext den Kontext belastet – nur ihre Ausgabe kostet Token.

Damit kann ein Skill beliebig viel Wissen bündeln, ohne jede Anfrage zu verteuern.

C. Abgrenzung zu verwandten Mechanismen

Table I ordnet Skills in das Umfeld der Agentenkonfiguration ein. Entscheidend ist der Unterschied zwischen *immer geladenem* Kontext (Regeldateien), *bei Bedarf geladenem* Wissen (Skills) und *deterministisch ausgeführten* Prüfungen (Hooks, Skripte).

TABLE I
MECHANISMEN ZUR STEUERUNG VON CODING-AGENTEN

Mechanismus	Ladeverhalten	Geeignet für
CLAUDE.md / AGENTS.md	immer, jede Anfrage	wenige Regeln, die jede Aufgabe betreffen
Skill	bei passender Aufgabe	Abläufe, Fachwissen, Prüfungsskripte
Hook	bei Ereignis, deterministisch	Pflichtprüfungen, die nie vergessen werden dürfen
MCP-Werkzeug	Beschreibung immer, Aufruf bei Bedarf	Zugriff auf externe Systeme
Subagent	eigener Kontext	breite Suche, Isolation langer Ausgaben

III. METHODIK

Die Arbeit folgt einem *Rapid Review*: Gesucht wurde im Oktober 2026 in arXiv, Konferenzbänden (ICLR, NeurIPS, ICSE, IEEE S&P) sowie in technischen Berichten von Werkzeugherstellern und Branchenstudien. Eingeschlossen wurden Arbeiten ab 2023 mit messbaren Aussagen zu Skills, Kontextdateien, Werkzeugauswahl, Kontextlänge, Rückkopplungsschleifen, Produktivität oder Sicherheit von Coding-Agenten. Da das Forschungsfeld sehr jung ist, sind viele Quellen Preprints oder Herstellerberichte; ihre Evidenzstärke wird im Text jeweils benannt. Ergänzend wurde eine Fallstudie an einer proprietären C++-Spiel-Engine (ZadeEngine) durchgeführt, deren Methodik in Section V beschrieben ist.

IV. ERGEBNISSE DER LITERATURANALYSE

A. Wirksamkeit von Skills

Die bislang umfangreichste Messung, *SkillsBench*, prüft 84 Aufgaben aus 11 Domänen in je drei Varianten (ohne Skill, mit kuratiertem Skill, mit selbstgeneriertem Skill) über 7.308

Durchläufe [9]. Kuratierte Skills erhöhen die Erfolgsrate im Mittel um 16,2 Prozentpunkte. Die Spannweite ist jedoch groß: +4,5 Prozentpunkte in der Softwareentwicklung, +51,9 in der Medizin, und 16 der 84 Aufgaben werden *schlechter*. Drei Befunde sind für die Praxis zentral: (1) Selbstgenerierte Skills bringen im Mittel keinen Nutzen – Modelle können das Wissen, von dem sie profitieren, nicht zuverlässig selbst verfassen. (2) Fokussierte Skills mit zwei bis drei Modulen schlagen umfassende Dokumentation. (3) Kleinere Modelle mit Skills erreichen größere Modelle ohne Skills.

Shaposhnikov et al. werten 500 reale Skills mit 1.000 abgeleiteten Aufgaben über 19 Agent-Modell-Konfigurationen aus und zeigen, dass ein Skill das Verhalten deutlich verändert, Modelle sich aber stark darin unterscheiden, wie treu sie seinen Anweisungen folgen [10]. Jiang et al. analysieren 8.135 Durchläufe und finden, dass Skills vor allem als *prozedurale Anker* wirken (65,7 % der erfolgreichen Fälle), während reine Wissensinjektion nur 4,5 % ausmacht [11]. Ein Skill hilft also weniger als Lexikon denn als verlässlicher Arbeitsablauf.

B. Aktivierung: das unterschätzte Problem

Ein Skill wirkt nur, wenn er geladen wird. In Vercels Evaluierung zu Next.js-16-APIs wurde der Skill in 56 % der Fälle nie aufgerufen; ein komprimierter Dokumentationsindex von 8 KB direkt in `AGENTS.md` erreichte 100 % Erfolg, der Skill höchstens 79 % [12]. Die Auswahl verschlechtert sich zudem mit der Zahl der Alternativen: Bei Jiang et al. fällt die Trefferquote der Skill-Auswahl von 29,6 % bei 5 Skills auf 3,3 % bei 100 Skills [11]. Ein ähnliches Muster zeigt sich bei Werkzeugen: Wenn nur die relevanten MCP-Werkzeuge per Retrieval vorausgewählt werden, steigt die Auswahlgenauigkeit von 13,62 % auf 43,13 % bei halbierten Prompt-Token [13].

Daraus folgt: Die Beschreibung ist die wichtigste Zeile eines Skills; sie muss konkrete Auslöser nennen. Pflichtwissen, das bei *jeder* Aufgabe gilt, gehört zusätzlich als kurzer Verweis in den immer geladenen Kontext.

C. Kontextbudget und Regeldateien

Mehr Kontext ist nicht automatisch besser. Gloaguen et al. zeigen an 138 realen Aufgaben, dass Kontextdateien wie `AGENTS.md` die Erfolgsrate von Coding-Agenten *senken* und die Inferenzkosten um über 20 % erhöhen; Agenten befolgen die Zusatzanweisungen gewissenhaft und explorieren dadurch mehr als nötig [14]. Die Autoren empfehlen, nur minimale Anforderungen aufzuschreiben. Das deckt sich mit Messungen zur Befolgung vieler gleichzeitiger Anweisungen: Selbst die besten Modelle befolgen bei 500 Anweisungen nur 68 %, mit einer Tendenz zugunsten früher genannter Anweisungen [15]. Unabhängig davon sinkt die Leistung mit wachsender Eingabelänge bereits bei einfachen Aufgaben (*Context Rot*) [16], und Informationen in der Mitte langer Kontexte werden schlechter genutzt als am Anfang oder Ende [17].

Für Skills bedeutet das: Wichtigstes zuerst, keine Erklärungen dessen, was das Modell ohnehin weiß, und Detailwissen

in Referenzdateien auslagern, die nur bei Bedarf gelesen werden [18].

D. Rückkopplungsschleifen schlagen Anweisungen

Anweisungen allein garantieren keine Qualität; Prüfwerkzeuge mit Rückmeldung an das Modell wirken stärker. Blyth et al. lassen GPT-4o Code iterativ anhand statischer Analyse (Bandit, Pylint) verbessern: Sicherheitsprobleme sinken von über 40 % auf 13 %, Lesbarkeitsverstöße von über 80 % auf 11 % und Zuverlässigkeitswarnungen von über 50 % auf 11 % [19]. Für C++ zeigen Tran et al. an 3,52 Millionen produktiven Code-Änderungen, dass KI-generierter Code überdurchschnittlich viele Kopier- und Allokationskosten, Kopplungsprobleme und handgeschriebene Schleifen statt Standardalgorithmen enthält, was 5–8 % mehr Rechenressourcen kostet; gezieltes, taxonomiebasiertes Feedback senkte die betroffenen Warnungen um 11,1 % [20]. Allgemeiner belegen *Self-Refine* [21] und *Reflexion* [22], dass Modelle mit Rückmeldung über mehrere Runden deutlich bessere Ergebnisse erzielen. Wie stark die Gestaltung der Schnittstelle zwischen Agent und Umgebung zählt, zeigt SWE-agent: Speziell entworfene Befehle mit kompakter Ausgabe verbessern die Lösungsrate realer GitHub-Issues erheblich [23].

Ein Skill sollte daher nicht nur beschreiben, wie guter Code aussieht, sondern Skripte mitliefern, die Abweichungen messen, und eine Schleife vorschreiben: prüfen, korrigieren, erneut prüfen.

E. Token-Ökonomie und Subagenten

In Anthropics Multi-Agenten-Recherchesystem erklärt die Menge verbrauchter Token allein 80 % der Leistungsunterschiede; das System übertrifft einen Einzelagenten um 90,2 %, benötigt aber etwa das 15-Fache an Token einer normalen Unterhaltung [24]. Subagenten lohnen sich daher für breite, parallelisierbare Erkundung, deren Zwischenergebnisse im eigenen Kontext bleiben – nicht für Aufgaben, die mit wenigen gezielten Suchen lösbar sind. Weil bei jedem Gesprächsschritt der gesamte Verlauf erneut verarbeitet wird, sparen gezielte Suche statt Volltextlesen, Teil-Lesen großer Dateien, Skriptausgaben mit Begrenzung und kurze Berichte direkt Kosten.

F. Sicherheit

Skills führen Anweisungen und Code mit dem Vertrauen des Agenten aus. Liu et al. untersuchen 31.132 Skills aus zwei öffentlichen Marktplätzen: 26,1 % enthalten mindestens eine Schwachstelle (Prompt Injection, Datenabfluss, Rechteauserweiterung, Lieferkettenrisiken), 5,2 % zeigen Muster, die auf Absicht schließen lassen, und Skills mit Skripten sind 2,12-mal häufiger betroffen als reine Anweisungs-Skills [25]. Hinzu kommen die bekannten Risiken generierten Codes selbst: Rund 40 % der von Copilot erzeugten Programme in sicherheitsrelevanten Szenarien waren verwundbar [26], und Coding-Agenten erzeugen messbare „Security Debt“ [27].

V. FALLSTUDIE: ZADEENGINE

A. Ausgangslage

ZadeEngine ist eine proprietäre C++20-Spiel-Engine (2.688 C++-Dateien, rund 676.000 Zeilen ohne Drittanbieter-Code) mit strikter Modulschichtung (Runtime, Developer, Editor, Programs) und einer bereits vorhandenen Regel-datei `CLAUDE.md` mit 133 Zeilen. Ziel war, dass beliebige KI-Agenten Code auf dem Niveau erfahrener Engine-Entwickler:innen schreiben: performant in heißen Pfaden, sauber geschichtet, modern und mit stetigem Rückbau von Altlasten.

B. Architektur der Lösung

Die Befunde aus Abschnitt 4 wurden in vier Ebenen übersetzt (Table II):

TABLE II
EBENEN DER SKILL-ARCHITEKTUR IN DER FALLSTUDIE

Ebene	Umsetzung
Immer-Kontext	3 zusätzliche Zeilen in <code>CLAUDE.md</code> , kurzer Verweis in <code>AGENTS.md</code> für andere Werkzeuge
Skill	<code>aaa-engine-engineering</code> : 172 Zeilen mit Senior-Maßstab, Ablauf, Prüfschleife, Gotcha-Tabelle und Format für Strukturwarnungen; 6 Referenzdateien
Skripte	Strukturscan (Größe, Hot-Path-Kosten, Kopien, Legacy-Idiome, Duplikate, Schichtverletzungen) und kuratierter <code>clang-tidy</code> -Lauf mit 51 Prüfregeln
Hooks	<code>UserPromptSubmit</code> gibt bei passenden Anfragen einen kurzen Skill-Verweis; <code>PostToolUse</code> prüft nach C++-Änderungen die geänderten Zeilen

Die Gotcha-Tabelle des Skills übernimmt die von Tran et al. gemessenen Schwächen (Kopien, Allokationen, handgeschriebene Schleifen, Kopplung) direkt als Prüfliste [20]. Ein zweiter, kleiner Skill `token-efficient-workflow` (78 Zeilen) beschreibt sparsames Arbeiten in großen Repositories. Beide liegen einmalig unter `.agents/skills/` und sind per Verzeichnisverknüpfung für Claude Code sichtbar, sodass es nur eine Quelle gibt.

C. Messwerte der Werkzeuge

Der Strukturscan analysiert die gesamte Engine in 11 s und meldete unter anderem 136 duplizierte Codeblöcke, darunter eine doppelt implementierte Material-Texturlade-logik in Editor und Runtime, 57 mutmaßliche Pro-Frame-Kosten und 35 teure Parameterkopien. Erste Versionen erzeugten Fehlalarme (Prosa-Kommentare als „toter Code“, eingebettete Drittanbieter-Header); sie wurden durch Stichproben am Quelltext gefunden und behoben – ein Prüfskript ist selbst Code, der Tests braucht. Der Hook läuft in 0,15 s je Änderung. Die Kompilierdatenbank des Build-Systems wurde bei jedem Build überschrieben und enthielt nur 187 Einträge; nach einer Korrektur mit Selbsttest führt sie Einträge zusammen und umfasst 1.014 Übersetzungseinheiten. Erst dadurch kann `clang-tidy` die Kernmodule prüfen (drei Übersetzungseinheiten in 10,8 s, 15 Funde). Ein Fund – `sizeof` auf einen Zeiger – erwies sich bei Durchsicht

als beabsichtigt: Statische Analyse liefert Hinweise, keine Urteile.

D. Vergleichsmessung mit und ohne Skill

Vier typische Aufgaben wurden je einmal mit und ohne Skill von unabhängigen Agenteninstanzen desselben Modells (Claude Opus 5.5) bearbeitet: eine Partikel-Optimierung, das Aufräumen einer 2.800-Zeilen-Datei mit zwei parallelen `switch`-Blöcken, eine Funktion, die eine Schichtverletzung (Runtime ruft Editor) provoziert, und ein Präfix-Filter, der jeden Frame läuft. Je Aufgabe wurden fünf vorab festgelegte, per Skript geprüfte Kriterien bewertet (Table III).

TABLE III
ERFÜLLTE PRÜFKRITERIEN UND KOSTEN JE KONFIGURATION

Aufgabe	mit Skill	ohne Skill
Partikel-System (SoA)	5/5	5/5
RenderSettings aufräumen	5/5	4/5
Runtime/Editor-Schichtung	5/5	4/5
HUD-Präfixfilter	5/5	4/5
Summe	20/20	17/20
Token je Lauf (Mittel)	64.048	52.025
Dauer je Lauf (Mittel)	123 s	80 s
Antwortlänge (Wörter, Mittel)	633	528

Beide Konfigurationen lösten die Kernprobleme; das Basismodell erkannte etwa die Schichtverletzung auch ohne Skill. Die Unterschiede lagen dort, wo Projektwissen nötig ist: Mit Skill verwiesen die Agenten auf die bestehende Einstellungstabelle des Projekts statt eine zweite anzulegen, entdeckten das vorhandene Partikelmodul und meldeten die drohende Doppelimplementierung als Strukturwarnung, nutzten die echte Job-System-API und schrieben Tests im Testformat des Projekts. Der Preis: rund 23 % mehr Token, 54 % mehr Laufzeit und 20 % längere Antworten. Das entspricht dem kleinen, aber positiven Effekt, den SkillsBench für die Softwareentwicklung misst [9], und führte unmittelbar zu einer Nachschärfung des Skills (Längenbegrenzung der Berichte).

Die Messung ist bewusst als Fallbeispiel zu lesen: ein Lauf je Zelle, ein Modell, Kriterien vom Autor festgelegt. Zudem war der Hook während der Messung aktiv und gab auch einem Basislauf eine Rückmeldung, und die Regeldatei des Projekts kann in den Kontext beider Konfigurationen gelangt sein; beides verringert den gemessenen Unterschied eher.

E. Aktivierung und bedingtes Routing

Die Wirksamkeitsmessung setzt einen geladenen Skill voraus; sie belegt nicht, dass ein Agent ihn selbstständig auswählt. Deshalb wurden für beide Projekt-Skills die Beschreibungen um konkrete deutsche und englische Auslöser sowie Abgrenzungen ergänzt. Die Regeldateien verlangen das Laden vor dem Planen passender Aufgaben. Ein zusätzlicher `UserPromptSubmit`-Hook liefert bei passenden Anfragen einen kurzen Hinweis, bleibt bei Dokumentararbeit, kurzen Begriffsfragen und reinen Build-

Aufträgen jedoch still. Reine Suche oder Lektüre löst den Engineering-Skill nicht aus; große Dateien, mehrere Module oder Kostenfragen können den Workflow-Skill auslösen. Die Hook-Schnittstelle fügt diesen Hinweis als Kontext hinzu [28]; das tatsächliche Laden bleibt eine Entscheidung des Agenten.

Die lokale Routingprüfung umfasst 16 ursprüngliche Anfragen und 18 ergänzende Grenzfälle. Alle 34 wurden wie erwartet zugeordnet; darunter sind Anfragen ohne Skill und Aufgaben, die beide Skills benötigen. Neun automatisierte Regressionstests prüfen außerdem die Hook-Ausgabe und die Auswertungslogik. Diese Zahlen beschreiben die deterministische Zuordnung im kuratierten Testsatz, keine Aktivierungsrate eines Modells und keine unabhängige Schätzung für neue Anfragen.

Der überarbeitete Modelltest beobachtet höchstens drei Agentenrunden mit lesenden Werkzeugen. Er zählt nur einen erfolgreichen Skill-Aufruf oder eine vollständige Lektüre der `SKILL.md` mit erfolgreicher Werkzeugantwort. Namensnennungen, fehlgeschlagene Aufrufe, nicht sichtbare Skills, Zeitüberschreitungen und API-Fehler dürfen die Aktivierungsrate nicht verfälschen. Eine neue Claude-Code-Sitzung (Version 2.1.288) registrierte beide Skills, der Verfügbarkeitslauf endete jedoch mit HTTP 429 wegen des Wochenlimits. Die erneute Modellmessung ist daher offen; eine Verbesserung der Aktivierungsrate wird nicht behauptet. Die im früheren Arbeitsprotokoll genannte Basislinie von 0 % lässt sich ohne Rohdaten und mit der damaligen Fehlerbehandlung nicht belastbar bestätigen.

VI. LEITFADEN: SKILLS RICHTIG BAUEN UND EINSETZEN

Dieser Abschnitt fasst Literatur, Herstellerdokumentation [8], [18], [29] und Fallstudie zu einer Arbeitsgrundlage für Teams zusammen. Er ist so geschrieben, dass er direkt als interne Richtlinie übernommen werden kann.

A. Wohin gehört welches Wissen?

Der häufigste Entwurfsfehler ist nicht ein schlechter Skill, sondern Wissen am falschen Ort. Table IV ordnet typische Inhalte dem passenden Mechanismus zu.

TABLE IV
ENTSCHEIDUNGSHILFE FÜR DEN ABLAGEORT VON WISSEN

Das Wissen ...	gehört in
gilt für jede Aufgabe und ist in wenigen Zeilen sagbar (Build-Befehl, Schichtregel)	CLAUDE.md / AGENTS.md
ist ein Ablauf oder Fachwissen für eine Aufgabenfamilie	Skill
darf nie vergessen werden (Formatierung, Pflichtprüfung)	Hook oder CI
braucht Zugriff auf ein externes System (Ticket, Datenbank)	MCP-Werkzeug
erfordert breite Suche mit viel Zwischenausgabe	Subagent
betrifft nur die aktuelle Aufgabe	Prompt

```

---
name: datenbank-migration
description: Plant und prueft Schemamigrationen der PostgreSQL-Datenbank des Bestellsystems. Verwenden bei neuen
  Tabellen, Spaltenaenderungen oder Indizes und wenn der Nutzer "Migration", "Schema" oder "DB-Update" sagt.
  Nicht fuer reine Leseabfragen.
license: Proprietaer, nur intern
metadata:
  owner: team-plattform
  version: "1.2.0"
---

# Datenbank-Migration
Ziel in einem Satz: Migrationen, die ohne Datenverlust vor- und zurueckrollen.

## Vorrang                                <- Projektregeln (AGENTS.md) gewinnen bei Widerspruch
## Ablauf                                <- nummerierte Checkliste, die der Agent abhakt
1. Bestehende Migrationen und Schema lesen
2. Migration mit Vorwaerts- und Rueckwaertsschritt schreiben
3. `python scripts/pruefen.py <datei>` ausfuehren, Funde beheben, Schritt 3 wiederholen
## Regeln                                <- jede Regel mit Begruendung ("weil ...")
- Spalten nie direkt loeschen, weil laufende Versionen sie noch lesen: erst entkoppeln, dann entfernen.
## Gotchas                                <- beobachtete Wiederholungsfehler, je Zeile Fehler -> Korrektur
## Ausgabeformat                          <- was der Bericht enthaelt, wie lang er ist
## Referenzen                             <- je Datei: wann lesen
- references/sperrern.md: bei Tabellen ueber 10 Mio. Zeilen

Listing 2. Vorlage für eine SKILL.md (Kommentare mit Pfeil sind Erläuterungen, nicht Teil der Datei)

```

Ein Skill ersetzt also keine Regeldatei und keinen Hook; er ergänzt sie. Pflichtwissen gehört zusätzlich als einzelzeiliger Verweis in den Immer-Kontext, weil Skills nicht zuverlässig ausgelöst werden [12].

B. Verzeichnisstruktur

Ein Skill deckt genau *eine* Aufgabenfamilie ab, etwa „Datenbankmigrationen“ oder „C++-Engine-Code“, nicht „alles zum Backend“. Listing 1 zeigt eine bewährte Struktur.

```

datenbank-migration/
+-- SKILL.md          Pflicht: Kopf + Anweisungen
+-- references/       Nachschlagewissen,
|   +-- sperrern.md   je Datei ein Thema
|   +-- rollback.md
+-- scripts/         ausfuehrbare Pruefungen
|   +-- pruefen.py
|   +-- test_pruefen.py
+-- assets/          Vorlagen, Konfiguration
+-- evals/
|   +-- evals.json    Testaufgaben
+-- README.md        fuer Menschen
+-- LICENSE           Nutzungsrechte

```

Listing 1. Verzeichnisstruktur eines Skills

Der Agent liest nur SKILL.md und die dort verlinkten Dateien; README.md richtet sich an Menschen (Zweck, Installation, Pflege) und kostet dem Agenten keine Token. Der Ordnername muss mit dem Feld name übereinstimmen. Skills liegen einmal im Repository unter .agents/skills/ und werden für Werkzeuge mit eigenem Pfad (Claude Code: .claude/skills/) per Verknüpfung eingebunden – eine Kopie je Werkzeug würde auseinanderlaufen.

C. Kopfdaten und Beschreibung

Table V fasst die Felder zusammen. Pflicht sind nur name und description; alle anderen sind optional [8].

TABLE V
FELDER IM KOPF EINER SKILL.md

Feld	Regel
name	1–64 Zeichen, nur a-z, 0-9 und Bindestrich, gleich dem Ordnernamen; keine reservierten Wörter wie „claude“ oder „anthropic“
description	höchstens 1.024 Zeichen, dritte Person, beschreibt <i>was</i> und <i>wann</i>
license	Lizenz oder Vertraulichkeitshinweis
compatibility	nur bei besonderen Voraussetzungen (Python-Version, Netzwerk)
metadata	Besitzer, Version und weitere Schlüssel für eigene Werkzeuge
allowed-tools	vorab erlaubte Werkzeuge; experimentell
Claude-Code-Zusätze	when_to_use, paths (nur bei passenden Dateien), disable-model-invocation (nur manuell), context: fork (eigener Subagent), hooks

Die Beschreibung entscheidet allein darüber, ob der Skill geladen wird. Bewährt hat sich die Formel **Was + Wann + Auslöser + Abgrenzung**:

- **Schwach:** „Hilft bei Datenbanken.“
- **Stark:** „Plant und prüft Schemamigrationen der PostgreSQL-Datenbank des Bestellsystems. Verwenden bei neuen Tabellen, Spaltenänderungen oder Indizes und wenn der Nutzer ‚Migration‘, ‚Schema‘ oder ‚DB-Update‘ sagt. Nicht für reine Leseabfragen.“

Das Wichtigste gehört an den Anfang. In Claude Code werden description und when_to_use zusammen nach 1.536 Zeichen abgeschnitten, und die Liste aller Skill-Beschreibungen erhält nur etwa 1 % des Kontextfensters; bei vielen Skills werden die Beschreibungen deshalb gekürzt [29]. Viele Skills verschlechtern so nicht nur die Auswahl [11], sondern auch die Beschreibungen selbst.

D. Aufbau der Anweisungen

Listing 2 zeigt eine Vorlage für den Rumpf. Die Reihenfolge ist Absicht: Nach einer Kontextverdichtung behält Claude Code nur die ersten 5.000 Token jedes aufgerufenen Skills [29], und Modelle befolgen früh genannte Anweisungen zuverlässiger [15]. Das Wichtigste steht deshalb oben, Nachschlagewissen unten oder in Referenzen.

Für den Schreibstil gelten sechs Regeln:

- 1) **Imperativ und konkret:** „Führe `pruefen.py` aus“ statt „man könnte prüfen“.
- 2) **Begründen statt befehlen:** Eine Regel mit „weil“ überträgt das Modell auf neue Fälle; gehäufte Großbuchstaben-MUSTs wirken starr und verdrängen einander.
- 3) **Freiheitsgrad passend wählen:** Bei riskanten Abläufen (Migrationen, Releases) exakte Befehle vorgeben; bei Entwurfsaufgaben Heuristiken und Ziele [18].
- 4) **Eine Bezeichnung je Begriff:** nicht abwechselnd „Feld“, „Spalte“, „Attribut“.
- 5) **Nichts Zeitabhängiges:** statt „ab Juli neue API“ einen Abschnitt „Alte Muster“.
- 6) **Pfade mit Schrägstrich** (`scripts/pruefen.py`), damit sie auf allen Systemen funktionieren.

E. Skripte und Referenzen

Skripte sind der wirksamste Teil eines Skills, weil sie Qualität *messen* statt sie zu beschreiben [19]. Ihr Code kostet keine Token, ihre Ausgabe schon. Daraus folgen Regeln:

- Im Skill eindeutig sagen, ob ein Skript **ausgeführt** oder **als Referenz gelesen** werden soll.
- Standardmäßig nur lesend; schreibende Aktionen nur mit ausdrücklicher Option.
- Kompakte, begrenzte Ausgabe (`--limit`, Filter, optional JSON), Fundstellen als `datei:zeile`.
- Eindeutige Exit-Codes (0 = sauber, 1 = Funde, 2 = Bedienfehler) und hilfreiche Fehlermeldungen statt Abstürzen.
- Möglichst nur Standardbibliothek; sonst Abhängigkeiten im Feld `compatibility` nennen.
- Keine unbegründeten Schwellwerte; jede Konstante mit kurzer Begründung.
- Kein Netzwerkzugriff und keine Zugangsdaten ohne zwingenden Grund.
- Selbst testen: absichtlich fehlerhafte Beispiele müssen gefunden werden, echter Code muss stichprobenartig auf Fehlalarme geprüft werden. In der Fallstudie fielen so drei Fehlalarmquellen auf, bevor der Skill eingesetzt wurde.

Referenzdateien behandeln je ein Thema, sind nur eine Ebene tief von `SKILL.md` verlinkt (bei tieferer Verschachtelung lesen Agenten Dateien oft nur teilweise) und erhalten ab etwa 100 Zeilen ein Inhaltsverzeichnis [18].

F. Richtig nutzen im Alltag

Für Entwickler:innen, die Skills anwenden, gelten diese Regeln:

- 1) **Bei klarer Aufgabe den Skill direkt aufrufen** (/name oder Nennung im Prompt), statt auf die automatische Auslösung zu hoffen.
- 2) **Eine Aufgabe je Sitzung.** Fremde Themen in einer neuen Sitzung beginnen, damit alter Kontext nicht mitbezahlt wird und nicht stört [16].
- 3) **Ergebnisse an Prüfungen messen**, nicht an der Überzeugungskraft der Antwort; wahrgenommene und tatsächliche Wirkung fallen auseinander [4].
- 4) **Wiederholte Fehler melden.** Jeder zweite gleiche Fehler wird eine Gotcha-Zeile.
- 5) **Kosten über das Modell steuern.** Kleinere Modelle mit gutem Skill erreichen größere Modelle ohne Skill [9]; für Routine reicht oft das günstigere Modell.
- 6) **Werkzeugfunde beurteilen.** Fehlalarme dürfen mit Begründung bestehen bleiben.

G. Anti-Muster

Table VI listet, wie Skills *nicht* eingesetzt werden sollten, jeweils mit Folge und Alternative.

H. Lebenszyklus und Freigabe im Unternehmen

Skills sind Code und brauchen denselben Lebenszyklus:

- 1) **Bedarf belegen:** wiederkehrende Fehler ohne Skill sammeln.
- 2) **Testaufgaben zuerst:** mindestens drei realistische Aufgaben mit prüfbaren Kriterien.
- 3) **Minimaler Entwurf:** nur so viel, dass die Testaufgaben bestehen.
- 4) **Messen:** mit und ohne Skill, mehrere Läufe, alle eingesetzten Modelle; neben Qualität auch Token und Laufzeit. In der Fallstudie zeigte erst diese Messung, dass der Skill die Antworten um 20 % verlängert.
- 5) **Review und Freigabe** nach der Checkliste unten.
- 6) **Ausrollen** über eine interne Registry oder das Repository.
- 7) **Beobachten:** Wird der Skill ausgelöst? Welche Fehler wiederholen sich?
- 8) **Pflegen:** Gotchas ergänzen, Version erhöhen, Änderungen gegen die Testaufgaben prüfen.
- 9) **Stilllegen**, wenn der Ablauf entfällt oder der Skill nicht mehr messbar hilft.

Freigabe-Checkliste:

- Name gleich Ordnername; Beschreibung mit Was, Wann, Auslösern und Abgrenzung.
- `SKILL.md` unter 500 Zeilen, Wichtigstes oben, keine zeitabhängigen Aussagen.
- Referenzen eine Ebene tief, je ein Thema, Inhaltsverzeichnis ab 100 Zeilen.
- Skripte gelesen, getestet, standardmäßig nur lesend, ohne Zugangsdaten.
- Mindestens drei Testaufgaben mit Vergleich gegen eine Basislinie.
- Besitzer, Version und Lizenz bzw. Vertraulichkeit eingetragen.
- Pflichtprüfungen zusätzlich als Hook oder in der CI.
- Keine inhaltliche Überschneidung mit bestehenden Skills.

TABLE VI
ANTI-MUSTER BEIM EINSATZ VON SKILLS

Anti-Muster	Folge	Besser
Skill als Ablage für alles (Wiki-Kopie, komplette API-Doku)	verwässerte Anweisungen, hohe Kosten	2–3 fokussierte Module, Rest in Referenzen [9]
Vage Beschreibung („Hilft bei Code“)	Skill wird nicht geladen	Was + Wann + Auslöser + Abgrenzung
Viele ähnliche Skills	falsche Auswahl, gekürzte Beschreibungen [11]	zusammenlegen, klare Grenzen
Pflichtprüfung nur im Skill	wird in einem Teil der Fälle vergessen [12]	zusätzlich Hook oder CI
Lange Regeldatei statt Skill	niedrigere Erfolgsrate, über 20 % Mehrkosten [14]	Regeldatei kurz, Details in Skills
Erklären, was das Modell ohnehin weiß	Token und Ablenkung	nur Projekt- und Domänenwissen
Vom Modell erzeugter Skill ohne Prüfung	im Mittel kein Nutzen [9]	aus echten Fehlern kuratieren
Starre MUST-Ketten ohne Begründung	schlechte Übertragung auf neue Fälle	Regel plus „weil“
Tief verschachtelte Referenzen	unvollständiges Lesen	eine Ebene, Inhaltsverzeichnis
Ungeprüfte Skills aus Marktplätzen	26,1 % verwundbar [25]	interne Registry mit Review
Zugangsdaten oder Kundendaten im Skill	Datenabfluss	Umgebungsvariablen, Secret-Store
Skripte mit ungefilterter Ausgabe	voller Kontext, höhere Kosten	Begrenzung, Zusammenfassung
Skill ohne Testaufgaben	Wirkung unbekannt	Evals mit Basislinie, mehrere Modelle

VII. VERWANDTE ARBEITEN

Die Idee wiederverwendbarer Fähigkeiten geht auf Agenten wie *Voyager* zurück, der eine wachsende Bibliothek ausführbarer Fertigkeiten aufbaut [30]. SWE-agent zeigt, dass die Schnittstelle zwischen Agent und Rechner ein eigener Gestaltungsgegenstand ist [23]. Zur Rolle von Entwickler:innen beschreiben Bird et al. die Verschiebung vom Schreiben zum Prüfen von Code [31]. Arbeiten zu Kontextdateien [14], Skill-Benchmarks [9], [10], [11] und Skill-Sicherheit [25] bilden die unmittelbare Grundlage dieser Arbeit.

VIII. SCHLUSSFOLGERUNG UND AUSBLICK

Agent Skills sind ein wirksames, werkzeugübergreifendes Mittel, um KI-Agenten das Wissen großer Projekte zugänglich zu machen – aber nur unter Bedingungen. Sie helfen, wenn sie von Menschen kuratiert, fokussiert und als Arbeitsablauf statt als Lexikon geschrieben sind (RQ2). Ihre größte Schwäche ist die unzuverlässige Aktivierung; ihre größte Stärke entfalten sie im Zusammenspiel mit kurzem Immer-Kontext, deterministischen Prüfskripten und Hooks (RQ1, RQ3). Im Unternehmen müssen sie wie Code behandelt werden: versioniert, getestet, gemessen und als Lieferkettenrisiko abgesichert (RQ4). Der Leitfaden in Section VI übersetzt diese Befunde in konkrete Regeln für Struktur, Beschreibung, Anweisungen, Skripte, Nutzung und Freigabe.

Für den Einstieg in einem Team genügen fünf Schritte: wiederkehrende Fehler sammeln, drei Testaufgaben formulieren, einen kleinen Skill nach Listing 2 schreiben, ihn mit und ohne Skill messen und Pflichtprüfungen zusätzlich als Hook absichern.

Offen sind vor allem drei Fragen: wie sich die Aktivierung von Skills ohne Kontextkosten zuverlässig machen lässt,

wie Skills mit dem Code, den sie beschreiben, automatisch aktuell bleiben, und wie sich ihr Nutzen in großen, proprietären Codebasen über längere Zeiträume messen lässt. Die Fallstudie liefert dafür einen ersten, bewusst kleinen Datenpunkt.

REFERENCES

- [1] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot,” *arXiv preprint arXiv:2302.06590*, 2023.
- [2] E. Paradis, K. Grey, Q. Madison, D. Nam, A. Macvean, and others, “How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2025.
- [3] Z. K. Cui, M. Demirer, S. Jaffe, L. Musolf, S. Peng, and T. Salz, “The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers,” 2025.
- [4] J. Becker, N. Rush, E. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” *arXiv preprint arXiv:2507.09089*, 2025.
- [5] DORA, “State of AI-assisted Software Development,” technical report, 2025.
- [6] W. Harding, “AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones,” technical report, 2025.
- [7] Anthropic, “Equipping Agents for the Real World with Agent Skills,” 2025.
- [8] Agent Skills Project, “Agent Skills Specification,” 2025.
- [9] X. Li, Y. Liu, W. Chen, B. You, Z. Di, and others, “SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks,” *arXiv preprint arXiv:2602.12670*, 2026.
- [10] M. Shaposhnikov, N. Fortuin, S. Stipich, M. I. Gorinova, A. Heineike, and R. Willoughby, “A Framework for Evaluating Agentic Skills at Scale,” *arXiv preprint arXiv:2606.17819*, 2026.
- [11] Z. Jiang et al., “Demystifying Agent Skills: Why They Work – Until They Don’t,” *arXiv preprint arXiv:2608.14036*, 2026.
- [12] Vercel, “AGENTS.md Outperforms Skills in Our Agent Evals,” 2026.
- [13] T. Gan and Q. Sun, “RAG-MCP: Mitigating Prompt Bloat in LLM Tool Selection via Retrieval-Augmented Generation,” *arXiv preprint arXiv:2505.03275*, 2025.

- [14] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, and M. Vechev, "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?," in *ICLR 2026 Workshop on AI with Recursive Self-Improvement*, 2026.
- [15] D. Jaroslawicz and others, "How Many Instructions Can LLMs Follow at Once?," *arXiv preprint arXiv:2507.11538*, 2025.
- [16] K. Hong, A. Troynikov, and J. Huber, "Context Rot: How Increasing Input Tokens Impacts LLM Performance," technical report, 2025.
- [17] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [18] Anthropic, "Skill Authoring Best Practices." 2026.
- [19] S. Blyth, S. A. Licorish, C. Treude, and M. Wagner, "Static Analysis as a Feedback Loop: Enhancing LLM-Generated Code Beyond Correctness," *arXiv preprint arXiv:2508.14419*, 2025.
- [20] M. Tran *et al.*, "Characterizing the Quality Profile of AI-Generated C++ in Production," *arXiv preprint arXiv:2608.06640*, 2026.
- [21] A. Madaan, N. Tandon, P. Gupta, and others, "Self-Refine: Iterative Refinement with Self-Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [22] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [23] J. Yang *et al.*, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [24] Anthropic, "How We Built Our Multi-Agent Research System." 2025.
- [25] Y. Liu *et al.*, "Agent Skills in the Wild: An Empirical Study of Security Vulnerabilities at Scale," *arXiv preprint arXiv:2601.10338*, 2026.
- [26] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 754–768.
- [27] M. Kozak, R. Zilouchian Moghaddam, and S. Sivaraman, "When Developer Aid Becomes Security Debt: A Systematic Analysis of Insecure Behaviors in LLM Coding Agents," *arXiv preprint arXiv:2507.09329*, 2025.
- [28] Anthropic, "Hooks reference." 2026.
- [29] Anthropic, "Extend Claude with Skills." 2026.
- [30] G. Wang *et al.*, "Voyager: An Open-Ended Embodied Agent with Large Language Models," *arXiv preprint arXiv:2305.16291*, 2023.
- [31] C. Bird, D. Ford, N. Forsgren, E. Kalliamvakou, and T. Zimmermann, "Taking Flight with Copilot," *Communications of the ACM*, vol. 66, no. 11, pp. 30–33, 2023, doi: 10.1145/3624745.